

CiT Backend Engineering Track

Week 3: Java Fundamentals — Part 1

Variables, Data Types, Operators, Control Flow & Methods

Detail	Information
Programme	CiT Backend Engineering Track
Phase	Phase 1 — Foundation (Weeks 1–8)
Week	Week 3
Instructor	Anderson
Duration	6 months 24 weeks
Tech Stack	Java 21, IntelliJ IDEA, Gradle

Confidential — For CiT Trainees Only

Opening Prayer

We begin, as always, by acknowledging that every gift of understanding comes from God. Please bow your heads as we invite His presence into this session.

Prayer of Dedication

Heavenly Father,

Today we begin learning Java — the language we will use to build real systems. We ask for clear minds as we encounter new syntax, patience when things do not compile the first time, and the wisdom to understand not just how to write code, but why.

As Proverbs 2:6 reminds us: "For the Lord gives wisdom; from His mouth come knowledge and understanding." Lord, let that wisdom flow through every concept we study today.

May we be diligent students who do not just copy but truly understand. May we encourage one another when we struggle. And may everything we build be offered as service to others.

In Jesus' name we pray,

Amen.

Scripture for the Week

"For the Lord gives wisdom; from His mouth come knowledge and understanding."
— **Proverbs 2:6 (NIV)**

Table of Contents

1. Welcome to Week 3
2. Your First Java Program
3. Variables & Data Types
4. Operators
5. Control Flow — if / else / switch
6. Loops
7. Methods
8. Introduction to Gradle
9. Key Takeaways, Homework & Exercises

1. Welcome to Week 3

Last week you built the engineering mindset — problem decomposition, clean code habits, Git, and professional communication. Now it is time to write real code.

Java is the language this entire track is built on. It powers the backend systems of some of the world's most demanding applications — from banking platforms to mobile money APIs to ride-hailing services. Learning it deeply is your most important investment in these first five weeks.

Weeks 3 to 5 are devoted entirely to Java Fundamentals. Do not rush. Every concept here is a building block. If you skip understanding something now, it will cost you triple the time later.

What to Expect in Week 3

By the end of this session, you will be able to:

- Write, compile, and run a Java program from scratch in IntelliJ
- Declare variables using the correct primitive and reference data types
- Use arithmetic, comparison, and logical operators correctly
- Control program flow with if/else and switch expressions
- Repeat logic using for, while, and enhanced for loops
- Define and call methods with parameters and return types
- Understand what Gradle does and how a Java project is structured

2. Before You Write Code — What is Java, Really?

Before we touch a keyboard, let us build a mental picture of what Java is and how it works. If you skip this, the code you write will feel like magic you are copying rather than a tool you understand.

2.1 What is Java?

Java is a programming language — a way of giving precise instructions to a computer. Just like English is a language humans use to communicate with each other, Java is a language programmers use to communicate with computers.

Java was created in 1995 and is still one of the most widely used languages in the world. The systems that process your mobile money transfer, the apps your bank runs, and many of the platforms powering Uganda's technology companies are built with Java.

2.2 What is the JDK?

JDK stands for Java Development Kit. Think of it as a toolbox.

When a carpenter builds a chair, they need a set of tools — a saw, a hammer, a measuring tape. They cannot build the chair with bare hands. A programmer is the same: before you can write and run Java code, you need a set of tools installed on your computer.

The JDK is that toolbox. It contains everything needed to write, compile, and run Java programs. You installed JDK 21 during Week 1 setup. Every time you write Java, the JDK is working behind the scenes.

2.3 What is the JVM?

JVM stands for Java Virtual Machine. This is one of Java's most powerful ideas, and it is worth understanding clearly.

Imagine you write a letter in English and want to send it to someone in Luganda, French, and Swahili at the same time. You would need a translator for each language. Now imagine instead you wrote the letter in a neutral middle language — a universal format — and then each recipient had their own translator that could read that format. You only write the letter once.

That is exactly how the JVM works. Your Java code gets converted into a universal middle language called bytecode. Every operating system — Windows, Mac, Linux — has its own JVM that can read that bytecode and run it. This is why Java programs run on any machine without being rewritten.

JDK vs JVM — Simple Summary

Term	Plain English
JDK (Java Development Kit)	The toolbox you install to write and build Java programs. Includes the compiler and the JVM.
JVM (Java Virtual Machine)	The engine that actually runs your Java program. It reads bytecode and executes it on whatever computer it is installed on.
Bytecode (.class file)	The universal middle format your Java code is converted into. Not human-readable, but the JVM understands it perfectly.

2.4 What Does "Compile" Mean?

When you write Java code, you are writing it for humans to read. The computer cannot execute it directly — it needs to be translated first. That translation process is called compiling.

Think of it like this: you write a recipe in English. Before a French chef can use it, someone must translate it into French. Compiling is that translation step — it converts your human-readable Java code into bytecode that the JVM can execute.

The tool that does this translation is called the compiler (javac). If you make a grammar mistake in your recipe — like writing "Add 2 cup flour" instead of "Add 2 cups flour" — the translator will refuse to continue until you fix it. The Java compiler works the same way: it will refuse to compile code that has syntax errors.

2.5 Creating Your First Java Project in IntelliJ

IntelliJ IDEA is the professional code editor we use. Follow these steps exactly:

1. Open IntelliJ IDEA → click "New Project"
2. Select Gradle as the build system, Java as the language, JDK 21
3. Name the project cit-java-week3 and click Create
4. IntelliJ creates a src/main/java folder — this is where all your Java code will live
5. Right-click src/main/java → New → Java Class → type Main and press Enter

2.6 Your First Java Program — HelloWorld

Type the following program exactly as it appears. Every character matters — Java is case-sensitive, so "main" and "Main" are different things.

```
public class Main {  
  
    public static void main(String[] args) {  
        System.out.println("Hello, CiT! Welcome to Java.");  
    }  
}
```

Press the green Play button at the top of IntelliJ. In the console at the bottom, you should see:

```
Hello, CiT! Welcome to Java.
```

Every Word Explained — No Assumptions

This is the most important code you will ever read. Understand every word before moving on.

Code	Plain English Explanation
public	This is an "access modifier". It means this class/method is accessible from anywhere. Think of it as saying the front door of this building is open to everyone.
class	A class is a blueprint or a template. Every Java program must live inside a class. You will understand classes much more deeply in Week 4.
Main	The name we gave to this class. The filename must also be called Main.java — Java requires the filename and class name to match exactly.
{ }	Curly braces mark the beginning and end of a block of code. Everything belonging to a class or method goes between its opening { and closing }.
static	This means the method belongs to the class itself, not to a specific object created from the class. For now, just accept that the main method must be static — we will revisit this in Week 4.
void	This means the method does not give back any result when it finishes. Like asking someone to "throw the rubbish away" — they do the job, but they do not hand you anything back.

<code>main</code>	This is the special starting point of every Java program. When you run a Java program, the JVM looks specifically for a method named "main" to start execution. The name is not a suggestion — it must be exactly "main".
<code>String[] args</code>	This is a parameter — information that can be passed into the method. <code>String[]</code> means "an array of text values". <code>args</code> is short for "arguments". For now, we are not using it, but Java requires it to be there.
<code>System.out.println(...)</code>	<code>System</code> is a built-in Java class. <code>out</code> is an output stream (think of it as a pipe to the screen). <code>println</code> means "print line" — it prints whatever is inside the brackets and then moves to the next line.
<code>"Hello, CiT!"</code>	This is a String literal — a piece of text written directly in the code. All text values in Java must be wrapped in double quotes.

What Just Happened? The Two Steps

When you pressed the Run button, IntelliJ silently did two things:

6. **Compiled:** It ran the Java compiler (`javac`) on your `Main.java` file. The compiler checked for errors and produced a `Main.class` bytecode file.
7. **Ran:** It started the JVM and told it to execute `Main.class`. The JVM found the `main` method and ran it, which called `System.out.println` and printed your message.

If you made a typo — say you wrote `sistem.out.println` — the compiler would stop at step 1 and show a red error. Nothing runs until the code compiles without errors.

3. Variables & Data Types

3.1 What is a Variable?

A variable is a named storage box in your computer's memory that holds a value. The name lets you find the box again later and read or change what is inside.

Imagine a shelf with labelled jars in a kitchen. One jar is labelled "sugar", another "salt". You know exactly where to find what you need and what each jar holds. A variable works exactly like that — it is a labelled jar in your computer's memory.

```
int age = 25;
```

This line does three things at once: it creates a box in memory, labels it age, and puts the value 25 inside it.

3.2 Why Do We Need Different Types?

Think about real storage containers. You would not store cooking oil in a sugar jar — the container should match what it holds. Programming types work the same way.

Different types of data require different amounts of memory. A whole number like 25 needs far less memory than a long decimal number like 3.14159265358979. By telling Java exactly what type of data a variable will hold, Java can allocate exactly the right amount of memory — no more, no less.

Types also prevent mistakes. If you declare a variable as an age (a whole number), Java will stop you the moment you try to accidentally store a name (text) in it. This catches bugs before your program ever runs.

3.3 Primitive Data Types

Java has 8 built-in basic types called primitive types. "Primitive" simply means they are the raw, fundamental building blocks — not objects, just pure values. You will use four of them in almost everything you build.

Type	What It Stores	Example
int	Whole numbers. Covers -2.1 billion to +2.1 billion. Use this for ages, counts, IDs, scores — anything that is never a fraction.	int age = 25;

long	Very large whole numbers that int cannot hold — like phone numbers, large account balances, or timestamps.	long balance = 5000000L;
double	Decimal (fractional) numbers with high precision. Use for money amounts, GPS coordinates, percentages, scientific values.	double price = 4999.99;
boolean	Stores exactly one of two values: true or false. Nothing else. Use for yes/no flags — isLoggedIn, hasPaidFees, isApproved.	boolean isPaid = false;
char	A single character — one letter, digit, or symbol. Must be wrapped in single quotes, not double quotes.	char grade = 'A';
byte	Very small whole numbers from -128 to 127. Used in file processing and networking where memory is critical.	byte level = 3;
short	Small whole numbers from -32,768 to 32,767. Rarely needed in modern Java.	short year = 2025;
float	Decimal numbers with less precision than double. The f suffix is required. Prefer double unless memory is extremely tight.	float rate = 3.14f;

3.4 String — Text in Java

String is how Java stores text — words, sentences, names, anything made of characters. Notice it starts with a capital S. That is because String is not a primitive type — it is a class (a more complex object). We will cover what classes are in Week 4, but for now, just know that String behaves slightly differently from the primitives above.

```
String name = "Victor";    // text values go inside double quotes
String city = "Kampala";
```

```
// Joining strings together is called concatenation — use the + sign
String greeting = "Hello, " + name + "!"; // "Hello, Victor!"

// String comes with built-in operations (methods):
int length = name.length(); // 6 — how many characters
String upper = name.toUpperCase(); // "VICTOR"
String lower = name.toLowerCase(); // "victor"
boolean hasV = name.contains("Vi"); // true
String part = name.substring(0, 3); // "Vic" — characters 0, 1, 2
```

Single quotes vs double quotes

char uses single quotes: `char grade = 'A';`

String uses double quotes: `String name = "Victor";`

Mixing them causes a compile error. `'Victor'` is not valid for a String in Java.

3.5 var — Let the Compiler Figure Out the Type

Since Java 10, you can use the keyword `var` instead of spelling out the type. The compiler looks at what value you are assigning and works out the type automatically. This is called type inference.

```
var name = "Victor"; // compiler sees "Victor" and knows: this is a String
var age = 25; // compiler sees 25 and knows: this is an int
var price = 4999.99; // compiler sees 4999.99 and knows: this is a double
```

When to use var and when not to

Use `var` when the right-hand side makes the type completely obvious to anyone reading the code. Avoid it when it would leave a reader guessing what type a variable is.

```
var x = getUserData(); // BAD — what type does getUserData() return? Nobody knows.
var name = "Victor"; // GOOD — clearly a String.
```

Important: You MUST assign a value when using `var`. You cannot write `var age;` with no value — the compiler has nothing to infer from.

3.6 Naming Variables — Rules and Conventions

Java has strict rules (the compiler enforces these) and community conventions (the community enforces these through code review). Know the difference.

Rule / Convention	Example
RULE: Start with a letter, \$ or _	age ✓ firstName ✓ 3score X (cannot start with a number)
RULE: No spaces	transactionDate ✓ transaction date X
RULE: Case-sensitive	age and Age and AGE are three completely different variables
CONVENTION: camelCase for variables	accountBalance, isLoggedIn, studentName
CONVENTION: PascalCase for class names	TransactionService, UserAccount, BankStatement
CONVENTION: UPPER_SNAKE_CASE for constants	final double VAT_RATE = 0.18; final int MAX_RETRIES = 3;
CONVENTION: Always descriptive	accountBalance ✓ x X temp X data X

Exercise 1: Variables & Types

Create a new Java class called `StudentProfile`. Declare a variable for each piece of information below using the most appropriate type. Then print each one.

- The student's full name (text)
- The student's age (whole number)
- The student's GPA — e.g. 3.75 (decimal number)
- Whether the student has paid their fees — yes or no (true/false)
- The student's registration number — e.g. 2024001234 (very large whole number)
- The student's grade letter — e.g. A (single character)

Expected output (with your own values):

```
Name: Alice Nakato
Age: 22
GPA: 3.75
Fees paid: true
Reg number: 2024001234
Grade: A
```

To combine a label and a variable in one print statement, use:
`System.out.println("Name: " + name);`

4. Operators

An operator is a symbol that performs an action on one or more values and produces a result. You already know operators from mathematics — $+$, $-$, $\times$, $\div$. Java uses the same idea, with a few additions.

Think of operators as the verbs of a programming statement. Just as a sentence needs a verb to make something happen ("Mary runs", "he paid", "she approved"), a Java expression needs an operator to produce a result.

4.1 Arithmetic Operators

These perform mathematical calculations. They work on numbers and produce a number.

Operator	Name & Plain English	Example & Result
$+$	Addition — adds two numbers together	$5 + 3 \rightarrow 8$
$-$	Subtraction — takes the second number away from the first	$10 - 4 \rightarrow 6$
$*$	Multiplication — Java uses $*$ not $\times$ (no $\times$ key on a keyboard)	$3 * 7 \rightarrow 21$
$/$	Division — Java uses $/$ not $\div$. WARNING: see the trap below.	$10 / 4 \rightarrow 2$
$\%$	Modulo — gives you the REMAINDER after division. See explanation below.	$10 \% 3 \rightarrow 1$

What is Modulo (%)? — Explained Simply

The $\%$ operator is unfamiliar to most beginners. Here is how to understand it: Imagine you have 10 mangoes and you want to share them equally among 3 people. Each person gets 3 mangoes. But $3 \times 3 = 9$, so you have 1 mango left over. That leftover is the remainder.

```
10 % 3 → 1    // 3 goes into 10 three times (9), remainder is 1
10 % 5 → 0    // 5 goes into 10 exactly twice, no remainder
7 % 2 → 1     // 2 goes into 7 three times (6), remainder is 1
```

The most common real-world use of $\%$: checking if a number is even or odd.

```
if (number % 2 == 0) {
```

```
    System.out.println("Even"); // no remainder when divided by 2
} else {
    System.out.println("Odd"); // remainder of 1 when divided by 2
}
```

You will also use % to cycle through values — for example, distributing tasks round-robin, or checking if a transaction ID is divisible by a certain number.

Integer Division — A Trap That Catches Every Beginner

When you divide two whole-number (int) values in Java, the result is ALWAYS a whole number. The decimal part is silently thrown away — it is NOT rounded.

```
int a = 10;
int b = 3;
System.out.println(a / b); // Prints 3, NOT 3.333
// The .333 is simply discarded
```

To get the decimal result, tell Java to treat at least one number as a decimal (double). The simplest way is to cast one value:

```
System.out.println((double) a / b); // Now prints 3.3333333333333335
```

Casting means telling Java: "treat this int as a double for this calculation." The original variable is not changed — only the calculation uses the double version.

4.2 Comparison Operators

Comparison operators compare two values and answer a yes/no question. The answer is always a boolean — either true (yes) or false (no).

Quick reminder: a boolean is a data type that holds exactly two possible values: true or false. Nothing else. You will use booleans constantly with comparisons and conditions.

Operator	Plain English Question It Answers	Example → Result
==	Are these two values equal? (use double == for comparison, not single =)	5 == 5 → true
!=	Are these two values NOT equal?	5 != 3 → true
>	Is the left value greater than the right?	7 > 3 → true
<	Is the left value less than the right?	2 < 9 → true

>=	Is the left value greater than OR equal to the right?	5 >= 5 → true
<=	Is the left value less than OR equal to the right?	4 <= 3 → false

= vs == — The Most Common Mistake in All of Java

```
int age = 25;    // = means ASSIGNMENT: put 25 into the box called age
age == 25        // == means COMPARISON: is the value in age equal to 25?
```

Writing `if (age = 25)` instead of `if (age == 25)` is a syntax error in Java. The compiler will catch it. But mixing them up in your thinking causes bugs in every language — build the habit of saying "equals equals" out loud when you type `==`

4.3 Logical Operators

Logical operators let you combine multiple boolean conditions into one. Think of them as the words "AND", "OR", and "NOT" in everyday speech.

Logical Operators — Real World Examples

&& (AND) — BOTH conditions must be true:

```
// You can withdraw money only if you are verified AND have enough balance
if (isVerified && balance >= amount) { ... }
```

|| (OR) — AT LEAST ONE condition must be true:

```
// You can enter the system if you are an admin OR a staff member
if (isAdmin || isStaff) { ... }
```

! (NOT) — flips true to false and false to true:

```
// If the user is NOT logged in, redirect to the login page
if (!isLoggedIn) { redirectToLogin(); }
```

In plain English: `!` reads as "not". So `!isLoggedIn` reads as "not logged in".

4.4 Assignment & Compound Assignment Operators

The assignment operator `=` puts a value into a variable. Compound assignment operators are shortcuts that combine an operation with assignment — they save you from writing the variable name twice.


```
int score = 0;      // = : simple assignment — put 0 into score

score = score + 10; // the long way: read score, add 10, put it back
score += 10;        // shortcut for the same thing ✓
score -= 5;         // same as: score = score - 5
score *= 2;         // same as: score = score * 2
score /= 4;         // same as: score = score / 4
score++;           // add exactly 1 — very common in loops
score--;           // subtract exactly 1
```

5. Control Flow — if / else / switch

Every program you will ever write needs to make decisions. Without the ability to make decisions, code is just a list of instructions that always does the same thing — not very useful.

Control flow is the order in which your program's instructions are executed. By default, Java runs your code top to bottom, line by line. Control flow statements let you change that — jumping over certain lines, repeating others, or choosing between different paths based on conditions.

5.1 What is a Condition?

A condition is a question your program asks that has a yes or no answer. In Java, that answer is always a boolean — either true or false. Conditions are almost always built using the comparison and logical operators you learned in Section 4.

Real-world conditions translated into Java

Everyday Question	Java Condition
Does the user have enough money?	<code>balance >= withdrawAmount</code>
Is the user logged in?	<code>isLoggedIn == true</code> or just: <code>isLoggedIn</code>
Is the user NOT logged in?	<code>!isLoggedIn</code>
Is the amount valid AND user verified?	<code>amount > 0 && isVerified</code>
Is the user an admin OR a manager?	<code>isAdmin isManager</code>

5.2 if / else if / else

An if statement says: "if this condition is true, run this block of code." You can chain additional conditions with else if, and provide a fallback with else.

```
int balance = 5000;
int withdrawAmount = 8000;

if (withdrawAmount <= 0) {
    // This block runs if the amount is 0 or negative
}
```

```
System.out.println("Invalid amount. Must be greater than zero.");
} else if (withdrawAmount > balance) {
    // This block runs if the first condition was false AND this one is true
    System.out.println("Insufficient funds.");
} else {
    // This block runs if ALL conditions above were false
    balance -= withdrawAmount;
    System.out.println("Withdrawal successful. New balance: " + balance);
}
```

Four Rules to Memorise

- The condition goes inside parentheses () — it must evaluate to true or false
- The code to run goes inside curly braces {} — always use braces even for a single line; missing braces create bugs that are nearly invisible
- Only ONE branch executes — Java checks from the top; the moment it finds a true condition, it runs that block and skips everything else
- else is your safety net — it runs when no condition above was true. It is optional, but always think about whether you need one.

5.3 The Ternary Operator — A One-Line if/else

When you need to choose between two values based on a condition, the ternary operator lets you write the whole thing in one line. The name "ternary" means it takes three parts.

```
// Long form using if/else:
String status;
if (balance >= 0) {
    status = "Active";
} else {
    status = "Overdrawn";
}

// Ternary form — exactly the same result, one line:
// condition ? value-if-true : value-if-false
String status = balance >= 0 ? "Active" : "Overdrawn";
```

Read it as: "Is balance >= 0? If yes, use "Active". If no, use "Overdrawn"." Use the ternary only for simple value choices — not for complex logic.

5.4 switch Expressions (Java 14+)

When you have one variable and many possible values to check, a chain of else if statements becomes repetitive. The switch expression is designed exactly for this situation.

Think of switch like a phone menu: "Press 1 for Balance, press 2 for Transfer, press 3 for Statement..." Each number (case) maps to a specific action.

```
String day = "MONDAY";

String type = switch (day) {
    case "MONDAY", "TUESDAY", "WEDNESDAY", "THURSDAY", "FRIDAY" -> "Weekday";
    case "SATURDAY", "SUNDAY" -> "Weekend";
    default -> "Unknown"; // default runs if no case matches
};

System.out.println(day + " is a " + type); // MONDAY is a Weekday
```

The -> arrow means "produce this value for this case". The default case is like the else — it catches anything that did not match.

Exercise 2: Control Flow — Mobile Money Validator

Write a Java program that simulates a mobile money transfer. Declare these variables at the top:

```
int senderBalance = 150000;
int transferAmount = 50000;
boolean senderIsVerified = true;
boolean receiverExists = true;
```

Write if/else logic that checks every condition and prints a clear message for each outcome:

- If the amount is 0 or negative → "Error: Transfer amount must be greater than zero."
- If the sender is not verified → "Error: Sender account is not verified."
- If the receiver does not exist → "Error: Receiver not found."
- If the balance is less than the transfer amount → "Error: Insufficient funds."
- If all checks pass → "Transfer successful." and print the new sender balance

Test each error by changing the variables. For example, set senderIsVerified = false and confirm your error message appears. Then fix it and test the next one.

6. Loops

Imagine you are a bank teller and 500 customers come in today. Each one wants their balance checked. Without a loop, you would write the same "check balance" code 500 times. With a loop, you write it once and tell the computer to repeat it 500 times.

A loop is a way to repeat a block of code multiple times without writing it multiple times. This is one of the most powerful ideas in all of programming.

6.1 What is an Array?

Before we look at loops, we need to understand arrays — because loops and arrays work together constantly.

An array is a fixed-size list of values of the same type, stored in order. Think of it as a row of labelled boxes. Instead of creating five separate variables `score1`, `score2`, `score3`..., you create one array called `scores` that holds all five values.

```
// Declaring an array of 5 integers:
int[] scores = {85, 92, 47, 76, 100};

// Accessing individual values by their position (index):
// Positions always start at 0, not 1 — this trips up EVERY beginner
System.out.println(scores[0]); // 85 ← first item
System.out.println(scores[1]); // 92 ← second item
System.out.println(scores[4]); // 100 ← fifth item (last)
```

Arrays start at index 0, not 1

In Java (and almost every programming language), positions in an array start counting from 0. An array with 5 items has positions 0, 1, 2, 3, 4. Trying to access position 5 causes an `ArrayIndexOutOfBoundsException` — one of the most common beginner errors. Always remember: first item is `[0]`, last item is `[length - 1]`.

6.2 The for Loop — When You Know the Count

Use a for loop when you know exactly how many times to repeat something — for example, going through every item in an array.

```
// for ( start ; condition to keep going ; update after each round )
for (int i = 0; i < 5; i++) {
    System.out.println("Round " + i);
}
```

```
// Prints: Round 0, Round 1, Round 2, Round 3, Round 4
```

Anatomy of a for Loop — Every Part Explained

Part	Plain English Explanation
<code>int i = 0</code>	Initialisation: create a counter variable called <code>i</code> and start it at 0. This runs ONCE before the loop begins.
<code>i < 5</code>	Condition: before every round, check this. If it is true, run the loop body. If it is false, stop. Here: keep going as long as <code>i</code> is less than 5.
<code>i++</code>	Update: after every round, add 1 to <code>i</code> . This is the same as <code>i = i + 1</code> . Without this, <code>i</code> never changes and the loop runs forever (an infinite loop).
<code>{ ... }</code>	The loop body: the code to repeat. It runs from top to bottom each time the condition is true.

Using a for loop to process every score in an array:

```
int[] scores = {85, 92, 47, 76, 100};

for (int i = 0; i < scores.length; i++) {
    System.out.println("Score " + i + ": " + scores[i]);
}

// scores.length gives the number of items in the array (5 here)
```

6.3 The Enhanced for Loop — The Cleaner Way to Read a List

When you want to go through every item in an array and you do not need to know the position (index), use the enhanced for loop. It is cleaner and harder to get wrong.

```
String[] students = {"Alice", "Bob", "Charles", "Diana"};

// for ( type variableName : arrayName )
for (String student : students) {
    // student takes the value of each item one by one
    System.out.println("Welcome, " + student + "!");
}

// Read as: "for each student in students, do this"
```

6.4 The while Loop — When You Do Not Know the Count

Use a while loop when you do not know in advance how many times to repeat — the loop keeps going as long as a condition is true.

Real example: a PIN entry system. You keep asking for a PIN until the user gets it right or runs out of attempts. You cannot know in advance how many tries they will need.

```
int attempts = 0;
int maxAttempts = 3;
boolean pinCorrect = false;

while (attempts < maxAttempts && !pinCorrect) {
    attempts++;
    System.out.println("Attempt " + attempts + ": Please enter your PIN");
    // In a real program you would read the PIN here and check it
    // For now we simulate: assume they always get it wrong
}

if (!pinCorrect) {
    System.out.println("3 incorrect attempts. Account locked.");
}
```

6.5 break and continue — Controlling the Loop

Sometimes you need to exit a loop early or skip one round. `break` and `continue` give you that control.

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) continue; // skip number 5 — jump straight to i=6
    if (i == 8) break;    // stop the loop entirely when i reaches 8
    System.out.println(i); // prints 1, 2, 3, 4, 6, 7
}
```

Exercise 3: Loops — Student Grade Reporter

Start with this array of exam scores:

```
int[] scores = {85, 92, 47, 76, 100, 38, 66, 55, 91, 73};
```

Write a program that loops through every score and:

- Prints the score and its letter grade ($A \geq 80$, $B \geq 70$, $C \geq 60$, $D \geq 50$, $F < 50$) — use a switch expression for the grade

- Keeps a running count of how many students passed (score ≥ 50) and how many failed
- Keeps a running total to calculate the class average at the end

Expected output format:

```
Score: 85 Grade: A
Score: 92 Grade: A
Score: 47 Grade: F
...
Passed: 8 Failed: 2 Average: 72.3
```

Tip: Declare your passed, failed, and total counter variables BEFORE the loop. Update them INSIDE the loop. Print them AFTER the loop.

7. Methods

7.1 What Problem Do Methods Solve?

Imagine writing a program to process 100 transactions. Each transaction needs the same five validation steps: check the amount, verify the sender, check the balance, check the receiver, and log the result.

Without methods, you would copy those five steps 100 times — 500 lines of identical code. If you spot a bug in step 3, you fix it in 100 different places. If you miss even one, your program has a silent bug.

A method is a named, reusable block of code. You define it once and call it by name wherever you need it. Fix the bug once, and every call benefits automatically. This is the DRY principle in action.

7.2 Defining and Calling a Method

A method definition tells Java: "here is a block of code, give it this name, and it needs these inputs." A method call tells Java: "run that block of code now."

```
public class BankAccount {  
  
    // — METHOD DEFINITION  
  
    static double applyInterest(double balance, double rate) {  
        return balance + (balance * rate);  
    }  
  
    public static void main(String[] args) {  
        // — METHOD CALLS  
  
        double savings = applyInterest(100000, 0.12); // 112000.0  
        double current = applyInterest(500000, 0.08); // 540000.0  
        double business = applyInterest(2000000, 0.15); // 2300000.0  
  
        System.out.println("Savings after interest: UGX " + savings);  
        System.out.println("Current after interest: UGX " + current);  
        System.out.println("Business after interest: UGX " + business);  
    }  
}
```

Every Part of a Method Declaration Explained

Part	Plain English Explanation
static	This method belongs to the class itself, not to a specific object. For now, write static on all your methods — we will explain this fully when we reach OOP in Week 4.
double	The return type — what type of value this method hands back when it is done. Here, it gives back a decimal number. If the method does not give anything back, write void instead.
applyInterest	The method name — use camelCase and a verb phrase that describes what it does (applyInterest, validateUser, printReceipt). The name is how you call it later.
(double balance, double rate)	Parameters — the inputs the method needs to do its job. Each parameter has a type and a name. Think of them as the method's ingredients.
return ...	Sends a value back to whoever called the method. The value must match the return type. return also immediately ends the method — no code after it runs.

7.3 void Methods — Methods That Do, Not Give

Not every method needs to give back a value. Some methods simply perform an action — print something, save to a database, send an email. These use the return type void, which means "this method gives back nothing".

```
// void means: this method does something but does not hand back a result
static void printReceipt(String name, double amount, String type) {
    System.out.println("===== RECEIPT =====");
    System.out.println("Customer : " + name);
    System.out.println("Type    : " + type);
    System.out.println("Amount  : UGX " + amount);
    System.out.println("=====");
}

// Calling it — no variable needed because it returns nothing
printReceipt("Alice Nakato", 50000, "Mobile Money Transfer");
```

7.4 Method Overloading — Same Name, Different Inputs

Java allows you to have multiple methods with the same name, as long as their parameter lists are different. The compiler automatically picks the right version based on what you pass in. This is called overloading.

Think of it like a calculator app: whether you type "Add two numbers" or "Add three numbers", it is still an add operation — but it handles the inputs differently.

```
// Version 1: standard 1% fee
static double calculateFee(double amount) {
    return amount * 0.01;
}

// Version 2: custom rate — same name, different parameters
static double calculateFee(double amount, double rate) {
    return amount * rate;
}

// Version 3: premium customer gets half the fee
static double calculateFee(double amount, boolean isPremium) {
    return isPremium ? amount * 0.005 : amount * 0.01;
}

// Java knows which version to use based on what you pass:
calculateFee(50000);           // calls version 1
calculateFee(50000, 0.02);     // calls version 2
calculateFee(50000, true);     // calls version 3
```

Exercise 4: Methods — Build a Mini Calculator

Create a new Java class called `Calculator`. Define the following static methods — each must do exactly ONE thing:

- `int add(int a, int b)` — returns the sum
- `int subtract(int a, int b)` — returns a minus b
- `double multiply(double a, double b)` — returns the product
- `double divide(double a, double b)` — returns the result, but if b is 0 print an error message and return 0
- `boolean isEven(int n)` — returns true if n is even (hint: use the % operator)
- `int max(int a, int b, int c)` — returns the largest of the three numbers

In `main()`, call each method with real values and print the results clearly. For example: `System.out.println("5 + 3 = " + add(5, 3));`

8. Introduction to Gradle

8.1 What is a Build Tool?

When you write a Java program, running it involves many steps: compile all your source files, download any external libraries your code needs, run tests to make sure nothing is broken, and then package everything into a file that can be deployed to a server.

Doing all of this manually every single time would be slow, error-prone, and inconsistent between team members. A build tool automates these steps. You run one command and it handles everything.

Gradle is the build tool this track uses — and the one used by most professional Java teams today, including the teams that build Spring Boot applications at companies across East Africa.

8.2 What is a Dependency?

A dependency is an external library — code written by someone else that you want to use in your project.

Think of it like cooking. If a recipe calls for ready-made tomato sauce, you do not make the sauce from scratch — you buy it from the shop. In software, a dependency is that ready-made ingredient: code someone else wrote, tested, and published so that everyone can use it.

For example, when we start Spring Boot in Week 9, you will add Spring Boot as a dependency. Instead of building an entire web server framework from scratch (which would take years), you declare: "my project depends on Spring Boot" — and Gradle downloads and wires it in automatically.

8.3 What Gradle Does For You

Task	What It Means in Plain English
Compilation	Runs the Java compiler on all your .java files in the correct order. You never need to run javac manually.
Dependency management	Reads your list of dependencies and automatically downloads the correct versions from the internet into your project. No manual downloading or copying of .jar files.
Testing	Finds and runs all your unit tests, then produces a report telling you which passed and which failed.

Packaging	Bundles your compiled code and all its dependencies into a single .jar file that can be copied to any server and run.
Task automation	Any custom task you define — generating code, running migrations, building Docker images, etc.

8.4 Project Structure

Every Gradle Java project follows this standard folder structure. IntelliJ creates it for you automatically when you create a new Gradle project.

```
cit-java-week3/
├── build.gradle.kts    ← your build configuration — dependencies go here
├── settings.gradle.kts ← project name
├── gradle/
│   └── wrapper/       ← Gradle Wrapper (see below)
├── src/
│   ├── main/
│   │   └── java/      ← YOUR production code lives here
│   └── test/
│       └── java/      ← your tests live here
```

8.5 The build.gradle.kts File — Your Project's Recipe

This file is the heart of your Gradle project. It tells Gradle: what plugins to use, where to download dependencies from, and which dependencies your project needs.

```
plugins {
    application // tells Gradle this is a runnable application
}

repositories {
    mavenCentral() // Maven Central is the main library repository online
} // Gradle downloads dependencies from here

dependencies {
    // This is where you list your dependencies
    // We will add Spring Boot, JPA, and others here in later weeks
}

java {
    toolchain { languageVersion = JavaLanguageVersion.of(21) }
```

```
}    // tells Gradle to compile with Java 21

application {
    mainClass = "Main" // which class has the main() method to run
}
```

8.6 Essential Gradle Commands

Run these commands in the terminal inside your project folder. Always use `./gradlew` (with the dot-slash) — never just `gradle`.

Command	What It Does	When to Use
<code>./gradlew run</code>	Compiles your code and runs the application	Every time you want to test your program
<code>./gradlew build</code>	Compiles + runs tests + creates the <code>.jar</code> file	Before committing to Git or deploying
<code>./gradlew test</code>	Runs all unit tests and shows results	After every change to verify nothing broke
<code>./gradlew clean</code>	Deletes all compiled output in the <code>build/</code> folder	When the build behaves strangely
<code>./gradlew tasks</code>	Lists all available tasks you can run	To explore what is available

Always use `./gradlew` — not `gradle`

The `./gradlew` script (called the Gradle Wrapper) is included in every project. It automatically downloads and uses the exact version of Gradle that the project was built with.

Why does this matter? If your teammate uses Gradle 8.5 and you use Gradle 7.2, the build might behave differently. The wrapper ensures everyone on the team — and every server — uses the same version. This is standard professional practice.

9. Key Takeaways, Homework & Exercises

9.1 What You Learned This Week

- Java project structure with IntelliJ and Gradle
- Every word in a HelloWorld program and what it means
- All 8 primitive types and String — when to use each
- var for type inference (Java 10+)
- Arithmetic, comparison, logical, and compound assignment operators
- if / else if / else, ternary operator, and switch expressions (Java 14+)
- for, while, and enhanced for loops — and when to use each
- Methods: defining, calling, parameters, return types, and overloading
- Gradle project structure and the five essential Gradle commands

9.2 Homework

8. Complete all four in-class exercises if you did not finish them — push the code to a GitHub repository
9. Build a Java program called MoneyTransfer that applies all of this week's concepts: variables for sender/receiver/amount, validation with if/else, a loop to process multiple transfers from an array, and methods for each validation rule
10. Read the official Java documentation for String methods at docs.oracle.com — practise 5 methods you have not used yet
11. Preview Week 4: Look up what a Java class and object are — come with one question ready

Coming Next — Week 4: Object-Oriented Programming

Jim & Anderson will introduce you to Object-Oriented Programming — the paradigm that Java was built for. You will learn classes, objects, constructors, and the first two OOP pillars: encapsulation and inheritance. The concepts you mastered this week are the foundation everything else sits on.

Closing Reflection

"The beginning of wisdom is this: Get wisdom. Though it cost all you have, get understanding."

— Proverbs 4:7 (NIV)

Java will feel unfamiliar today. That is normal. Every professional you admire once sat exactly where you are sitting. Keep showing up, keep practising, and keep asking questions. The breakthrough always comes.

Glossary of Terms

Term	Definition
boolean	A data type that holds only true or false
Bytecode	The compiled output of a Java program (.class file) — platform-independent instructions the JVM executes
Compile	Converting human-readable source code into machine-executable bytecode
Data Type	Specifies what kind of value a variable holds and how much memory to reserve
Gradle	A build tool that automates compilation, dependency management, testing, and packaging
int	A primitive type for whole numbers between approximately -2.1 billion and 2.1 billion
JDK	Java Development Kit — the tools needed to write, compile, and run Java programs
JVM	Java Virtual Machine — the runtime that executes Java bytecode on any operating system
Method	A named, reusable block of code that performs a specific task
Method Overloading	Defining multiple methods with the same name but different parameter lists
Operator	A symbol that performs an operation on one or more values (e.g. +, >, &&)
Primitive Type	A basic built-in data type in Java: int, long, double, float, boolean, char, byte, short
String	A class representing a sequence of characters. Not a primitive type in Java.
Variable	A named storage location in memory that holds a value of a specific type
var	Java 10+ keyword for local type inference — the compiler deduces the type from the assigned value